Capítulo 03 - Infraestrutura e DevOps

3.1. Introdução à Infraestrutura do Sistema

A infraestrutura do Projeto Quimera foi projetada com três objetivos principais em mente: **portabilidade**, **reprodutibilidade** e **observabilidade**. Utilizamos containerização com Docker para garantir que o sistema funcione de forma idêntica em qualquer ambiente (desenvolvimento local, staging, produção). A orquestração é feita com Docker Compose para ambientes simples e pode ser migrada para Kubernetes quando necessário escalar horizontalmente. Todo o sistema é instrumentado com métricas, logs e traces para garantir visibilidade completa do seu funcionamento.

3.1.1. Princípios de Infraestrutura

1. Infrastructure as Code (IaC)

Toda a infraestrutura é definida em código (Docker Compose, Dockerfiles, scripts de configuração), versionada no Git e pode ser recriada a qualquer momento com um único comando. Isso elimina "configurações manuais" que são difíceis de reproduzir e propensas a erros.

2. Immutable Infrastructure

Containers são imutáveis. Uma vez criados, não são modificados. Se precisamos fazer uma mudança, criamos uma nova imagem e substituímos o container antigo. Isso garante consistência e facilita rollback.

3. Twelve-Factor App

Seguimos os princípios do Twelve-Factor App, especialmente:

- **Codebase:** Um codebase versionado, múltiplos deploys
- **Dependencies:** Dependências explícitas e isoladas
- **Config:** Configuração via variáveis de ambiente
- **Backing Services:** Serviços de suporte (DB, cache) tratados como recursos anexados
- **Build, Release, Run:** Separação estrita entre build, release e run
- **Processes:** Processos stateless e share-nothing
- **Port Binding:** Serviços expostos via port binding
- **Logs:** Logs como streams de eventos

4. Observability First

Desde o primeiro commit, o sistema emite métricas, logs estruturados e traces. Observabilidade não é uma adição posterior; é parte integrante do design.

3.2. Containerização com Docker

3.2.1. Estrutura de Diretórios

Plain Text

```
projeto-quimera/
├── docker/
│   ├── agente_batedor/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── agente_analista/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── agente_escriba/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── agente_execucao/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── agente_monitoramento/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── api_backend/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── orquestrador/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   └── frontend/
│       ├── Dockerfile
│       └── package.json
├── docker-compose.yml
├── docker-compose.dev.yml
├── docker-compose.prod.yml
├── .env.example
└── scripts/
    ├── build.sh
    ├── deploy.sh
    └── backup.sh
```

3.2.2. Dockerfile Base para Agentes Python

Criamos um Dockerfile base que todos os agentes herdam, reduzindo duplicação e garantindo consistência.

Plain Text

```
# docker/base/Dockerfile.python
FROM python:3.11-slim

# Metadados
LABEL maintainer="projeto-quimera@example.com"
LABEL version="1.0"

# Variáveis de ambiente
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

# Instalar dependências do sistema
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    curl \
    git \
    && rm -rf /var/lib/apt/lists/*

# Criar usuário não-root para segurança
RUN useradd -m -u 1000 quimera && \
    mkdir -p /app && \
    chown -R quimera:quimera /app

# Definir diretório de trabalho
WORKDIR /app

# Copiar requirements e instalar dependências Python
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Mudar para usuário não-root
USER quimera

# Comando padrão (será sobrescrito pelos agentes específicos)
CMD ["python", "-u", "main.py"]
```

3.2.3. Dockerfile do Agente Batedor

Plain Text

```
# docker/agente_batedor/Dockerfile
FROM projeto-quimera/python-base:latest

# Instalar dependências específicas do Agente Batedor
# (Selenium, undetected-chromedriver, etc.)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Instalar Chrome e ChromeDriver
USER root
RUN apt-get update && apt-get install -y --no-install-recommends \
    chromium \
    chromium-driver \
    && rm -rf /var/lib/apt/lists/*

USER quimera

# Copiar código-fonte
COPY --chown=quimera:quimera . .

# Healthcheck
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD python -c "import requests;
requests.get('http://localhost:8080/health')"

# Expor porta (para healthcheck e métricas)
EXPOSE 8080

# Comando de execução
CMD ["python", "-u", "agente_batedor/main.py"]
```

requirements.txt do Agente Batedor:

Plain Text

```
# docker/agente_batedor/requirements.txt
selenium==4.15.0
undetected-chromedriver==3.5.4
beautifulsoup4==4.12.2
lxml==4.9.3
redis==5.0.1
pydantic==2.5.0
python-dotenv==1.0.0
requests==2.31.0
```

```
prometheus-client==0.19.0
structlog==23.2.0
```

3.2.4. Dockerfile do Agente Analista

Plain Text

```
# docker/agente_analista/Dockerfile
FROM projeto-quimera/python-base:latest

# Instalar dependências de ML/AI
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Baixar modelos pré-treinados (para acelerar inicialização)
RUN python -c "from transformers import AutoModel, AutoTokenizer; \
    AutoTokenizer.from_pretrained('neuralmind/bert-base-portuguese-cased'); \
    AutoModel.from_pretrained('neuralmind/bert-base-portuguese-cased')"

# Copiar código-fonte
COPY --chown=quimera:quimera . .

# Healthcheck
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \
    CMD python -c "import requests;
requests.get('http://localhost:8081/health')"

EXPOSE 8081

CMD ["python", "-u", "agente_analista/main.py"]
```

requirements.txt do Agente Analista:

Plain Text

```
# docker/agente_analista/requirements.txt
torch==2.1.0
transformers==4.35.0
sentence-transformers==2.2.2
scikit-learn==1.3.2
opencv-python-headless==4.8.1.78
pillow==10.1.0
numpy==1.26.2
pandas==2.1.3
redis==5.0.1
pydantic==2.5.0
```

```
python-dotenv==1.0.0
prometheus-client==0.19.0
structlog==23.2.0
psycopg2-binary==2.9.9
pymongo==4.6.0
chromadb==0.4.18
```

3.2.5. Dockerfile da API Backend (FastAPI)

Plain Text

```
# docker/api_backend/Dockerfile
FROM python:3.11-slim

ENV PYTHONUNBUFFERED=1

# Instalar dependências
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar código
COPY --chown=1000:1000 . .

# Criar usuário não-root
RUN useradd -m -u 1000 quimera && chown -R quimera:quimera /app
USER quimera

# Healthcheck
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

EXPOSE 8000

# Usar uvicorn com workers
CMD ["uvicorn", "api_backend.main:app", "--host", "0.0.0.0", "--port",
"8000", "--workers", "4"]
```

requirements.txt da API Backend:

Plain Text

```
# docker/api_backend/requirements.txt
fastapi==0.104.1
uvicorn[standard]==0.24.0
pydantic==2.5.0
```

```
pydantic-settings==2.1.0
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
python-multipart==0.0.6
redis==5.0.1
psycopg2-binary==2.9.9
sqlalchemy==2.0.23
alembic==1.13.0
prometheus-fastapi-instrumentator==6.1.0
structlog==23.2.0
httpx==0.25.2
```

3.2.6. Dockerfile do Frontend (React)

Plain Text

```
# docker/frontend/Dockerfile

# Stage 1: Build
FROM node:20-alpine AS builder

WORKDIR /app

# Copiar package.json e instalar dependências
COPY package.json package-lock.json ./
RUN npm ci

# Copiar código-fonte e build
COPY . .
RUN npm run build

# Stage 2: Production
FROM nginx:alpine

# Copiar build do stage anterior
COPY --from=builder /app/build /usr/share/nginx/html

# Copiar configuração customizada do nginx
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Healthcheck
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD wget --quiet --tries=1 --spider http://localhost:80/health || exit 1

EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

nginx.conf:

Plain Text

```
# docker/frontend/nginx.conf
server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    # Compressão gzip
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_types text/plain text/css text/xml text/javascript
               application/x-javascript application/xml+rss
               application/json application/javascript;

    # Cache de assets estáticos
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # SPA: redirecionar todas as rotas para index.html
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Healthcheck endpoint
    location /health {
        access_log off;
        return 200 "OK\n";
        add_header Content-Type text/plain;
    }

    # Proxy para API backend
    location /api/ {
        proxy_pass http://api_backend:8000/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
```

```
        proxy_cache_bypass $http_upgrade;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # WebSocket para notificações
    location /ws/ {
        proxy_pass http://api_backend:8000/ws/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 86400;
    }
}
```

3.3. Orquestração com Docker Compose

3.3.1. Docker Compose Principal

YAML

```yaml
# docker-compose.yml
version: '3.8'

services:
  # ============================================
  # BANCOS DE DADOS
  # ============================================

  postgres:
    image: postgres:15-alpine
    container_name: quimera-postgres
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-quimera}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-senha_segura_aqui}
      POSTGRES_DB: ${POSTGRES_DB:-quimera_db}
      PGDATA: /var/lib/postgresql/data/pgdata
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./scripts/init-db.sql:/docker-entrypoint-initdb.d/init.sql
    ports:
```

```yaml
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U quimera"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - quimera-network
    restart: unless-stopped

  mongodb:
    image: mongo:7
    container_name: quimera-mongodb
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER:-quimera}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD:-senha_segura_aqui}
      MONGO_INITDB_DATABASE: ${MONGO_DB:-quimera_db}
    volumes:
      - mongodb_data:/data/db
      - ./scripts/init-mongo.js:/docker-entrypoint-initdb.d/init.js
    ports:
      - "27017:27017"
    healthcheck:
      test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - quimera-network
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    container_name: quimera-redis
    command: redis-server --requirepass ${REDIS_PASSWORD:-senha_segura_aqui}
--appendonly yes
    volumes:
      - redis_data:/data
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "--raw", "incr", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - quimera-network
    restart: unless-stopped
```

```yaml
  chromadb:
    image: chromadb/chroma:latest
    container_name: quimera-chromadb
    environment:
      CHROMA_SERVER_AUTH_CREDENTIALS: ${CHROMA_AUTH:-token_seguro_aqui}
      CHROMA_SERVER_AUTH_PROVIDER:
chromadb.auth.token.TokenAuthServerProvider
    volumes:
      - chromadb_data:/chroma/chroma
    ports:
      - "8100:8000"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/api/v1/heartbeat"]
      interval: 30s
      timeout: 10s
      retries: 3
    networks:
      - quimera-network
    restart: unless-stopped

  # ============================================
  # AGENTES
  # ============================================

  agente_batedor:
    build:
      context: .
      dockerfile: docker/agente_batedor/Dockerfile
    container_name: quimera-agente-batedor
    environment:
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      -
POSTGRES_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432
/${POSTGRES_DB}
      - LOG_LEVEL=${LOG_LEVEL:-INFO}
      - PROXY_ENABLED=${PROXY_ENABLED:-false}
      - PROXY_URL=${PROXY_URL:-}
    volumes:
      - ./logs/agente_batedor:/app/logs
      - /dev/shm:/dev/shm  # Shared memory para Chrome
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - quimera-network
```

```yaml
    restart: unless-stopped
    deploy:
      resources:
        limits:
          cpus: '1.0'
          memory: 1G
        reservations:
          cpus: '0.5'
          memory: 512M

  agente_analista:
    build:
      context: .
      dockerfile: docker/agente_analista/Dockerfile
    container_name: quimera-agente-analista
    environment:
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - 
POSTGRES_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432
/${POSTGRES_DB}
      - 
MONGODB_URL=mongodb://${MONGO_USER}:${MONGO_PASSWORD}@mongodb:27017/${MONGO_D
B}
      - CHROMADB_URL=http://chromadb:8000
      - CHROMADB_AUTH=${CHROMA_AUTH}
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - LOG_LEVEL=${LOG_LEVEL:-INFO}
    volumes:
      - ./logs/agente_analista:/app/logs
      - ./models:/app/models  # Modelos ML pré-treinados
    depends_on:
      postgres:
        condition: service_healthy
      mongodb:
        condition: service_healthy
      redis:
        condition: service_healthy
      chromadb:
        condition: service_healthy
    networks:
      - quimera-network
    restart: unless-stopped
    deploy:
      resources:
        limits:
          cpus: '2.0'
          memory: 4G
        reservations:
```

```yaml
          cpus: '1.0'
          memory: 2G

  motor_persuasao:
    build:
      context: .
      dockerfile: docker/motor_persuasao/Dockerfile
    container_name: quimera-motor-persuasao
    environment:
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - 
POSTGRES_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432
/${POSTGRES_DB}
      - LOG_LEVEL=${LOG_LEVEL:-INFO}
    volumes:
      - ./logs/motor_persuasao:/app/logs
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - quimera-network
    restart: unless-stopped

  agente_escriba:
    build:
      context: .
      dockerfile: docker/agente_escriba/Dockerfile
    container_name: quimera-agente-escriba
    environment:
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - 
POSTGRES_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432
/${POSTGRES_DB}
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - LOG_LEVEL=${LOG_LEVEL:-INFO}
    volumes:
      - ./logs/agente_escriba:/app/logs
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - quimera-network
    restart: unless-stopped
```

```
  agente_execucao:
    build:
      context: .
      dockerfile: docker/agente_execucao/Dockerfile
    container_name: quimera-agente-execucao
    environment:
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      -
POSTGRES_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432
/${POSTGRES_DB}
      - PROXY_ENABLED=${PROXY_ENABLED:-true}
      - PROXY_URL=${PROXY_URL}
      - LOG_LEVEL=${LOG_LEVEL:-INFO}
    volumes:
      - ./logs/agente_execucao:/app/logs
      - /dev/shm:/dev/shm
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - quimera-network
    restart: unless-stopped

  agente_monitoramento:
    build:
      context: .
      dockerfile: docker/agente_monitoramento/Dockerfile
    container_name: quimera-agente-monitoramento
    environment:
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      -
POSTGRES_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432
/${POSTGRES_DB}
      - PROXY_ENABLED=${PROXY_ENABLED:-true}
      - PROXY_URL=${PROXY_URL}
      - LOG_LEVEL=${LOG_LEVEL:-INFO}
    volumes:
      - ./logs/agente_monitoramento:/app/logs
      - /dev/shm:/dev/shm
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
```

```yaml
      - quimera-network
    restart: unless-stopped

  agente_aprendizagem:
    build:
      context: .
      dockerfile: docker/agente_aprendizagem/Dockerfile
    container_name: quimera-agente-aprendizagem
    environment:
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - POSTGRES_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}
      - LOG_LEVEL=${LOG_LEVEL:-INFO}
    volumes:
      - ./logs/agente_aprendizagem:/app/logs
      - ./models:/app/models
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - quimera-network
    restart: unless-stopped

  # ============================================
  # SERVIÇOS
  # ============================================

  orquestrador:
    build:
      context: .
      dockerfile: docker/orquestrador/Dockerfile
    container_name: quimera-orquestrador
    environment:
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - POSTGRES_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}
      - LOG_LEVEL=${LOG_LEVEL:-INFO}
    ports:
      - "8001:8001"
    volumes:
      - ./logs/orquestrador:/app/logs
    depends_on:
      postgres:
        condition: service_healthy
```

```yaml
      redis:
        condition: service_healthy
    networks:
      - quimera-network
    restart: unless-stopped

  api_backend:
    build:
      context: .
      dockerfile: docker/api_backend/Dockerfile
    container_name: quimera-api-backend
    environment:
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - 
POSTGRES_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432
/${POSTGRES_DB}
      - JWT_SECRET=${JWT_SECRET:-secret_jwt_super_seguro}
      - JWT_ALGORITHM=HS256
      - JWT_EXPIRATION=3600
      - LOG_LEVEL=${LOG_LEVEL:-INFO}
    ports:
      - "8000:8000"
    volumes:
      - ./logs/api_backend:/app/logs
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
      orquestrador:
        condition: service_started
    networks:
      - quimera-network
    restart: unless-stopped

  frontend:
    build:
      context: ./frontend
      dockerfile: ../docker/frontend/Dockerfile
    container_name: quimera-frontend
    ports:
      - "80:80"
    depends_on:
      - api_backend
    networks:
      - quimera-network
    restart: unless-stopped
```

```yaml
  # ==========================================
  # MONITORAMENTO E OBSERVABILIDADE
  # ==========================================

  prometheus:
    image: prom/prometheus:latest
    container_name: quimera-prometheus
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.path=/prometheus'
      - '--storage.tsdb.retention.time=30d'
      - '--web.console.libraries=/usr/share/prometheus/console_libraries'
      - '--web.console.templates=/usr/share/prometheus/consoles'
    volumes:
      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
      - prometheus_data:/prometheus
    ports:
      - "9090:9090"
    networks:
      - quimera-network
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: quimera-grafana
    environment:
      - GF_SECURITY_ADMIN_USER=${GRAFANA_USER:-admin}
      - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD:-admin}
      - GF_INSTALL_PLUGINS=redis-datasource
    volumes:
      - grafana_data:/var/lib/grafana
      - ./monitoring/grafana/dashboards:/etc/grafana/provisioning/dashboards
      -
./monitoring/grafana/datasources:/etc/grafana/provisioning/datasources
    ports:
      - "3000:3000"
    depends_on:
      - prometheus
    networks:
      - quimera-network
    restart: unless-stopped

  loki:
    image: grafana/loki:latest
    container_name: quimera-loki
    command: -config.file=/etc/loki/local-config.yaml
    volumes:
      - ./monitoring/loki-config.yml:/etc/loki/local-config.yaml
```

```yaml
      - loki_data:/loki
    ports:
      - "3100:3100"
    networks:
      - quimera-network
    restart: unless-stopped

  promtail:
    image: grafana/promtail:latest
    container_name: quimera-promtail
    command: -config.file=/etc/promtail/config.yml
    volumes:
      - ./monitoring/promtail-config.yml:/etc/promtail/config.yml
      - ./logs:/var/log/quimera
      - /var/lib/docker/containers:/var/lib/docker/containers:ro
    networks:
      - quimera-network
    restart: unless-stopped

# ============================================
# VOLUMES
# ============================================

volumes:
  postgres_data:
    driver: local
  mongodb_data:
    driver: local
  redis_data:
    driver: local
  chromadb_data:
    driver: local
  prometheus_data:
    driver: local
  grafana_data:
    driver: local
  loki_data:
    driver: local

# ============================================
# NETWORKS
# ============================================

networks:
  quimera-network:
    driver: bridge
    ipam:
```

```
      config:
        - subnet: 172.20.0.0/16
```

3.3.2. Arquivo de Variáveis de Ambiente

Bash

```bash
# .env.example
# Copie para .env e preencha com valores reais

# ==========================================
# BANCOS DE DADOS
# ==========================================

POSTGRES_USER=quimera
POSTGRES_PASSWORD=senha_super_segura_postgres_123
POSTGRES_DB=quimera_db

MONGO_USER=quimera
MONGO_PASSWORD=senha_super_segura_mongo_123
MONGO_DB=quimera_db

REDIS_PASSWORD=senha_super_segura_redis_123

CHROMA_AUTH=token_super_seguro_chroma_123

# ==========================================
# APIs EXTERNAS
# ==========================================

OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

# ==========================================
# PROXIES
# ==========================================

PROXY_ENABLED=true
PROXY_URL=http://usuario:senha@proxy.iproyal.com:12321

# ==========================================
# SEGURANÇA
# ==========================================

JWT_SECRET=chave_jwt_super_secreta_256_bits_minimo
JWT_ALGORITHM=HS256
JWT_EXPIRATION=3600
```

```
# ==============================================
# MONITORAMENTO
# ==============================================

GRAFANA_USER=admin
GRAFANA_PASSWORD=senha_grafana_segura

# ==============================================
# CONFIGURAÇÕES GERAIS
# ==============================================

LOG_LEVEL=INFO
ENVIRONMENT=production
```

3.3.3. Scripts de Build e Deploy

Script de Build:

Bash

```bash
#!/bin/bash
# scripts/build.sh

set -e

echo "🔨 Construindo imagens Docker do Projeto Quimera..."

# Build da imagem base Python
echo "📦 Construindo imagem base Python..."
docker build -t projeto-quimera/python-base:latest -f
docker/base/Dockerfile.python .

# Build de todos os agentes
AGENTES=("agente_batedor" "agente_analista" "motor_persuasao"
"agente_escriba" "agente_execucao" "agente_monitoramento"
"agente_aprendizagem")

for agente in "${AGENTES[@]}"; do
    echo "📦 Construindo $agente..."
    docker build -t projeto-quimera/$agente:latest -f
docker/$agente/Dockerfile .
done

# Build dos serviços
echo "📦 Construindo orquestrador..."
docker build -t projeto-quimera/orquestrador:latest -f
```

```
docker/orquestrador/Dockerfile .

echo "📦 Construindo API backend..."
docker build -t projeto-quimera/api_backend:latest -f
docker/api_backend/Dockerfile .

echo "📦 Construindo frontend..."
docker build -t projeto-quimera/frontend:latest -f
docker/frontend/Dockerfile ./frontend

echo "✅ Build concluído com sucesso!"
```

Script de Deploy:

Bash

```bash
#!/bin/bash
# scripts/deploy.sh

set -e

echo "🚀 Iniciando deploy do Projeto Quimera..."

# Verificar se .env existe
if [ ! -f .env ]; then
    echo "❌ Arquivo .env não encontrado!"
    echo "📝 Copie .env.example para .env e configure as variáveis."
    exit 1
fi

# Parar containers existentes
echo "🛑 Parando containers existentes..."
docker-compose down

# Build das imagens
echo "🔨 Construindo imagens..."
./scripts/build.sh

# Iniciar serviços
echo "🚀 Iniciando serviços..."
docker-compose up -d

# Aguardar healthchecks
echo "⏳ Aguardando serviços ficarem saudáveis..."
sleep 30

# Verificar status
echo "📊 Status dos serviços:"
```

```
docker-compose ps

# Executar migrações de banco de dados
echo "🗄️ Executando migrações de banco de dados..."
docker-compose exec api_backend alembic upgrade head

echo "✅ Deploy concluído com sucesso!"
echo "🌐 Frontend: http://localhost"
echo "🔧 API Backend: http://localhost:8000"
echo "📊 Grafana: http://localhost:3000"
echo "📈 Prometheus: http://localhost:9090"
```

Script de Backup:

Bash

```
#!/bin/bash
# scripts/backup.sh

set -e

BACKUP_DIR="./backups/$(date +%Y%m%d_%H%M%S)"
mkdir -p "$BACKUP_DIR"

echo "💾 Iniciando backup do Projeto Quimera..."

# Backup PostgreSQL
echo "📦 Backup PostgreSQL..."
docker-compose exec -T postgres pg_dump -U quimera quimera_db | gzip >
"$BACKUP_DIR/postgres.sql.gz"

# Backup MongoDB
echo "📦 Backup MongoDB..."
docker-compose exec -T mongodb mongodump --username quimera --password senha
--authenticationDatabase admin --archive | gzip >
"$BACKUP_DIR/mongodb.archive.gz"

# Backup Redis (RDB snapshot)
echo "📦 Backup Redis..."
docker-compose exec -T redis redis-cli --no-auth-warning -a senha SAVE
docker cp quimera-redis:/data/dump.rdb "$BACKUP_DIR/redis-dump.rdb"

# Backup ChromaDB
echo "📦 Backup ChromaDB..."
docker cp quimera-chromadb:/chroma/chroma "$BACKUP_DIR/chromadb"

# Backup de modelos ML
echo "📦 Backup de modelos ML..."
```

```
tar -czf "$BACKUP_DIR/models.tar.gz" ./models

# Backup de logs
echo "📦 Backup de logs..."
tar -czf "$BACKUP_DIR/logs.tar.gz" ./logs

echo "✅ Backup concluído: $BACKUP_DIR"

# Limpar backups antigos (manter últimos 7 dias)
find ./backups -type d -mtime +7 -exec rm -rf {} +
```

3.4. Monitoramento e Observabilidade

3.4.1. Configuração do Prometheus

YAML

```yaml
# monitoring/prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: 'projeto-quimera'
    environment: 'production'

# Alertmanager (opcional)
alerting:
  alertmanagers:
    - static_configs:
        - targets: []

# Regras de alerta
rule_files:
  - 'alerts.yml'

# Scrape configs
scrape_configs:
  # API Backend
  - job_name: 'api_backend'
    static_configs:
      - targets: ['api_backend:8000']
    metrics_path: '/metrics'

  # Orquestrador
  - job_name: 'orquestrador'
```

```yaml
    static_configs:
      - targets: ['orquestrador:8001']
    metrics_path: '/metrics'

  # Agentes
  - job_name: 'agente_batedor'
    static_configs:
      - targets: ['agente_batedor:8080']
    metrics_path: '/metrics'

  - job_name: 'agente_analista'
    static_configs:
      - targets: ['agente_analista:8081']
    metrics_path: '/metrics'

  # PostgreSQL Exporter
  - job_name: 'postgres'
    static_configs:
      - targets: ['postgres-exporter:9187']

  # Redis Exporter
  - job_name: 'redis'
    static_configs:
      - targets: ['redis-exporter:9121']

  # Node Exporter (métricas do host)
  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']

  # cAdvisor (métricas de containers)
  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']
```

3.4.2. Regras de Alerta

YAML

```yaml
# monitoring/alerts.yml
groups:
  - name: quimera_alerts
    interval: 30s
    rules:
      # Alta taxa de erro na API
      - alert: HighErrorRate
        expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.05
```

```
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "Alta taxa de erro na API ({{ $labels.instance }})"
          description: "Taxa de erro HTTP 5xx acima de 5% nos últimos 5
minutos."

      # Latência alta na API
      - alert: HighLatency
        expr: histogram_quantile(0.95,
rate(http_request_duration_seconds_bucket[5m])) > 1
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Latência alta na API ({{ $labels.instance }})"
          description: "P95 da latência acima de 1 segundo."

      # Banco de dados PostgreSQL down
      - alert: PostgreSQLDown
        expr: up{job="postgres"} == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "PostgreSQL está down"
          description: "PostgreSQL não está respondendo há mais de 1 minuto."

      # Redis down
      - alert: RedisDown
        expr: up{job="redis"} == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "Redis está down"
          description: "Redis não está respondendo há mais de 1 minuto."

      # Uso alto de memória
      - alert: HighMemoryUsage
        expr: (node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes)
/ node_memory_MemTotal_bytes > 0.9
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Uso alto de memória ({{ $labels.instance }})"
```

```
          description: "Uso de memória acima de 90%."

      # Uso alto de CPU
      - alert: HighCPUUsage
        expr: 100 - (avg by(instance)
(irate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Uso alto de CPU ({{ $labels.instance }})"
          description: "Uso de CPU acima de 80%."

      # Fila Redis crescendo indefinidamente
      - alert: RedisQueueGrowing
        expr: redis_list_length{list="eventos:PerfilColetado"} > 1000
        for: 10m
        labels:
          severity: warning
        annotations:
          summary: "Fila Redis crescendo"
          description: "Fila de eventos PerfilColetado tem mais de 1000
itens há 10 minutos."
```

3.4.3. Configuração do Loki

YAML

```yaml
# monitoring/loki-config.yml
auth_enabled: false

server:
  http_listen_port: 3100

ingester:
  lifecycler:
    address: 127.0.0.1
    ring:
      kvstore:
        store: inmemory
      replication_factor: 1
    final_sleep: 0s
  chunk_idle_period: 5m
  chunk_retain_period: 30s

schema_config:
  configs:
```

```yaml
    - from: 2023-01-01
      store: boltdb
      object_store: filesystem
      schema: v11
      index:
        prefix: index_
        period: 168h

storage_config:
  boltdb:
    directory: /loki/index

  filesystem:
    directory: /loki/chunks

limits_config:
  enforce_metric_name: false
  reject_old_samples: true
  reject_old_samples_max_age: 168h

chunk_store_config:
  max_look_back_period: 0s

table_manager:
  retention_deletes_enabled: true
  retention_period: 720h  # 30 dias
```

3.4.4. Configuração do Promtail

YAML

```yaml
# monitoring/promtail-config.yml
server:
  http_listen_port: 9080
  grpc_listen_port: 0

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  # Logs dos agentes
  - job_name: agentes
    static_configs:
      - targets:
```

```yaml
          - localhost
        labels:
          job: agentes
          __path__: /var/log/quimera/agente_*/*.log

  # Logs da API
  - job_name: api
    static_configs:
      - targets:
          - localhost
        labels:
          job: api
          __path__: /var/log/quimera/api_backend/*.log

  # Logs do orquestrador
  - job_name: orquestrador
    static_configs:
      - targets:
          - localhost
        labels:
          job: orquestrador
          __path__: /var/log/quimera/orquestrador/*.log

  # Logs de containers Docker
  - job_name: docker
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
        refresh_interval: 5s
    relabel_configs:
      - source_labels: ['__meta_docker_container_name']
        regex: '/(.*)'
        target_label: 'container'
      - source_labels: ['__meta_docker_container_log_stream']
        target_label: 'stream'
```

3.5. CI/CD Pipeline

3.5.1. GitHub Actions Workflow

YAML

```yaml
# .github/workflows/ci-cd.yml
name: CI/CD Pipeline

on:
```

```yaml
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  # ==============================================
  # TESTES
  # ==============================================
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: postgres
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432

      redis:
        image: redis:7
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 6379:6379

    steps:
      - name: Checkout código
        uses: actions/checkout@v3

      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'
          cache: 'pip'
```

```yaml
      - name: Instalar dependências
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install -r requirements-dev.txt

      - name: Lint com flake8
        run: |
          flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics
          flake8 . --count --exit-zero --max-complexity=10 --max-line-
length=127 --statistics

      - name: Type checking com mypy
        run: |
          mypy --ignore-missing-imports .

      - name: Testes unitários
        env:
          POSTGRES_URL: postgresql://postgres:postgres@localhost:5432/test_db
          REDIS_URL: redis://localhost:6379/0
        run: |
          pytest tests/unit -v --cov=. --cov-report=xml

      - name: Testes de integração
        env:
          POSTGRES_URL: postgresql://postgres:postgres@localhost:5432/test_db
          REDIS_URL: redis://localhost:6379/0
        run: |
          pytest tests/integration -v

      - name: Upload coverage para Codecov
        uses: codecov/codecov-action@v3
        with:
          files: ./coverage.xml
          flags: unittests
          name: codecov-umbrella

  # ============================================
  # BUILD E PUSH DE IMAGENS
  # ============================================
  build:
    needs: test
    runs-on: ubuntu-latest
    if: github.event_name == 'push'

    permissions:
      contents: read
```

```yaml
      packages: write

    strategy:
      matrix:
        component:
          - agente_batedor
          - agente_analista
          - motor_persuasao
          - agente_escriba
          - agente_execucao
          - agente_monitoramento
          - agente_aprendizagem
          - orquestrador
          - api_backend
          - frontend

    steps:
      - name: Checkout código
        uses: actions/checkout@v3

      - name: Setup Docker Buildx
        uses: docker/setup-buildx-action@v2

      - name: Login no GitHub Container Registry
        uses: docker/login-action@v2
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Extrair metadados
        id: meta
        uses: docker/metadata-action@v4
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}/${{
matrix.component }}
          tags: |
            type=ref,event=branch
            type=ref,event=pr
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=sha

      - name: Build e push
        uses: docker/build-push-action@v4
        with:
          context: .
          file: docker/${{ matrix.component }}/Dockerfile
```

```yaml
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  # ============================================
  # DEPLOY (apenas para branch main)
  # ============================================
  deploy:
    needs: build
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
      - name: Checkout código
        uses: actions/checkout@v3

      - name: Setup SSH
        uses: webfactory/ssh-agent@v0.7.0
        with:
          ssh-private-key: ${{ secrets.SSH_PRIVATE_KEY }}

      - name: Deploy para servidor de produção
        run: |
          ssh -o StrictHostKeyChecking=no ${{ secrets.SSH_USER }}@${{
secrets.SSH_HOST }} << 'EOF'
            cd /opt/projeto-quimera
            git pull origin main
            docker-compose pull
            docker-compose up -d
            docker-compose exec -T api_backend alembic upgrade head
          EOF

      - name: Verificar saúde dos serviços
        run: |
          sleep 30
          curl -f http://${{ secrets.SSH_HOST }}/health || exit 1
```

3.6. Conclusão do Capítulo

Este capítulo cobriu a infraestrutura completa do Projeto Quimera, incluindo:

- **Containerização:** Dockerfiles para todos os componentes
- **Orquestração:** Docker Compose completo com 15+ serviços

- **Monitoramento:** Prometheus, Grafana, Loki, Promtail
- **Alertas:** Regras de alerta para problemas críticos
- **CI/CD:** Pipeline completo com GitHub Actions
- **Scripts:** Build, deploy e backup automatizados

No próximo capítulo, mergulharemos na **modelagem de dados**, cobrindo o schema completo do PostgreSQL, estrutura do MongoDB e estratégias de indexação.

Próximo Capítulo: Capítulo 04 - Modelagem de Dados

Referências deste Capítulo:

- Docker Documentation: https://docs.docker.com/
- Docker Compose Documentation: https://docs.docker.com/compose/
- Prometheus Documentation: https://prometheus.io/docs/
- Grafana Documentation: https://grafana.com/docs/
- The Twelve-Factor App: https://12factor.net/
- GitHub Actions Documentation: https://docs.github.com/en/actions